

UNIVERSIDADE DE AVEIRO

Departamento de Eletrónica, Telecomunicações e Informática

Redes e Sistemas Autónomos

Cooperative Lane Merge

via Comunicação V2V

Sistema autónomo de apoio à inserção cooperativa em faixa de rodagem

Pedro Melo n.º 114208

6 de junho de 2026

Repositório: [\(inserir URL\)](#)

Vídeo da demo: [\(inserir URL / Discord\)](#)

Conteúdo

1	Introdução	2
2	Objetivos	2
2.1	Problema	2
2.2	Objetivo geral	3
2.3	Objetivos específicos	3
2.4	Síntese	3
3	Arquitetura	4
3.1	Visão geral em camadas	5
3.2	Nós e topologia	6
3.3	Mapa de tópicos Zenoh	6
3.4	Modelo de papéis	7
3.5	Tipos de mensagens trocadas	7
4	Implementação	8
4.1	Visão geral e a negociação num relance	8
4.2	O protocolo MergeProtocol	10
4.2.1	Lado do Merge Car (rampa)	11
4.2.2	Lado do veículo da via principal	12
4.3	Fundamentos geométricos e cinemáticos	15
4.4	Mensagens e consciência situacional	17
4.5	Orquestração, visualização e modo demo	18
5	Fluxo da Demonstração	19
5.1	Preparação	19
5.2	Timeline	19
5.3	Aleatoriedade e variáveis	19
5.4	Cenários	20
6	Resultados	20
7	Conclusão	20

1 Introdução

O aumento da densidade de tráfego nas entradas de autoestrada torna a inserção de veículos provenientes de rampas de acesso um dos pontos mais críticos para a fluidez e a segurança rodoviária. Tradicionalmente, esta manobra depende do julgamento individual de cada condutor, o que frequentemente origina travagens bruscas, paragens na rampa ou inserções forçadas que propagam ondas de congestionamento para a via principal.

Este projeto, desenvolvido no âmbito da unidade curricular de Redes e Sistemas Autónomos, explora como a comunicação Veículo-a-Veículo (V2V) pode substituir esse julgamento individual por uma **negociação cooperativa e distribuída**, sem qualquer entidade central de controlo. O cenário estudado é o de um veículo — o *Merge Car* (MC) — que se aproxima da via principal a partir de uma rampa de acesso e necessita de coordenar a sua inserção com os veículos que já circulam nessa via. Em vez de impor a sua entrada, o MC inicia um protocolo de coordenação de manobra em que os veículos afetados são identificados em tempo real, decidem cooperativamente um abrandamento controlado e concedem ao MC uma janela temporal segura para a inserção.

A solução assenta integralmente em mensagens normalizadas ETSI C-ITS: *Cooperative Awareness Messages* (CAM) para a consciência situacional periódica e *Manoeuvre Coordination Messages* (MCM) para a negociação da manobra. A pilha de comunicação é fornecida pelo Vanetza-NAP, o transporte de mensagens pelo middleware Zenoh, e toda a lógica dos veículos corre em containers Docker independentes, garantindo que cada veículo é um nó autónomo que só conhece a sua vizinhança através das CAMs que recebe. Uma dashboard web em tempo real permite observar a negociação e a manobra à medida que decorrem.

Este relatório descreve os objetivos do trabalho (Secção 2), a arquitetura do sistema (Secção 3), a implementação detalhada do protocolo e dos algoritmos de decisão (Secção 4), o fluxo da demonstração (Secção 5) e os resultados obtidos nos vários cenários testados (Secção 6), incluindo casos com dois veículos a inserir-se em simultâneo.

2 Objetivos

2.1 Problema

A inserção de um veículo proveniente de uma rampa de acesso na via principal é uma das manobras com maior potencial de conflito da circulação rodoviária. Sem cooperação entre veículos, o condutor que vem da rampa só dispõe da sua própria perceção visual para encontrar uma folga: quando esta não existe, vê-se obrigado a forçar a entrada, a travar bruscamente ou mesmo a parar no final da rampa à espera de espaço. Cada uma destas reações tem custos:

- **Inserção forçada** — obriga os veículos da via principal a travagens de emergência, comprometendo a segurança;
- **Paragem na rampa** — elimina a velocidade de aproximação necessária para uma inserção suave e cria risco de colisão traseira na própria rampa;
- **Travagens reativas** — propagam-se para montante sob a forma de ondas de choque,

degradando a fluidez de todo o troço.

A raiz do problema é a *falta de informação antecipada e de coordenação*: os veículos da via principal só percebem a intenção de inserção quando o veículo da rampa já está praticamente sobre o ponto de conflito, deixando pouca margem para uma resposta suave.

2.2 Objetivo geral

O objetivo central deste trabalho é demonstrar que a comunicação Veículo-a-Veículo (V2V) permite resolver este problema através de uma **negociação cooperativa, distribuída e antecipada**, sem qualquer entidade central de controlo e assente exclusivamente em mensagens normalizadas ETSI C-ITS (CAM e MCM). Pretende-se que o veículo da rampa — o *Merge Car* (MC) — anuncie a sua intenção com antecedência suficiente para que os veículos afetados se ajustem de forma coordenada e lhe abram uma janela temporal segura, substituindo a reação tardia por uma manobra planeada.

2.3 Objetivos específicos

Para concretizar o objetivo geral, definiram-se os seguintes objetivos específicos:

- O1. Consciência situacional contínua** — cada veículo difunde periodicamente a sua posição, velocidade, aceleração e dimensões via CAM, construindo uma visão local do tráfego sem qualquer configuração prévia da vizinhança.
- O2. Protocolo de negociação de manobra** — definir e implementar um protocolo distribuído sobre MCM que permita ao MC pedir autorização e aos restantes veículos responder cooperativamente.
- O3. Detecção de conflito** — determinar, em cada veículo da via principal e em tempo real, se irá efetivamente partilhar a zona de conflito com o MC, combinando a geometria da estrada com uma janela temporal de ocupação.
- O4. Abrandamento coordenado em cadeia** — quando há conflito, calcular uma velocidade-alvo viável e propagar o pedido de abrandamento aos veículos a montante, de modo a abrir a folga necessária sem travagens bruscas.
- O5. Garantia de segurança** — assegurar que a manobra nunca compromete a segurança: se a negociação não concluir a tempo, o MC recorre a um mecanismo de *stop-and-wait* no final da rampa e volta a tentar.
- O6. Suporte a múltiplos veículos a inserir-se** — estender o protocolo a cenários com dois veículos a fundir-se simultaneamente a partir da rampa.
- O7. Observabilidade** — disponibilizar uma visualização em tempo real da simulação e da troca de mensagens, para validação e demonstração do protocolo.

2.4 Síntese

A Tabela 1 resume como cada objetivo específico é concretizado na implementação, remetendo para a secção correspondente.

Objetivo	Mecanismo de concretização	Secção
O1 – Consciência situacional	Beaconing CAM a 100 ms + <code>NeighbourTable</code> com janela de 200 ms	4
O2 – Protocolo de negociação	6 subtipos de MCM (<code>MERGE_REQUEST</code> , <code>SLOW-DOWN_REQUEST/GRANT</code> , <code>MERGE_GRANT</code> , <code>MERGE_CONFIRMED</code> , <code>EXECUTION_STATUS</code>)	4
O3 – Detecção de conflito	Projeção geométrica (<code>project_t</code>) + sobreposição temporal entre a ocupação do MC e a entrada/saída do veículo na zona	4
O4 – Abrandamento em cadeia	Velocidade-alvo por busca binária (<code>_solve_target_speed</code>) e propagação de <code>SLOW-DOWN_REQUEST</code> ao veículo de trás	4
O5 – Garantia de segurança	Verificação cinemática de travagem (<code>can_brake_in_time</code>) e paragem em <code>stop_t</code> com retentativa	4
O6 – Múltiplos MC	Mesmo protocolo na rampa: o MC propaga <code>SLOW-DOWN_REQUEST</code> ao veículo atrás na rampa	4
O7 – Observabilidade	Ponte Zenoh→WebSocket (<code>bridge.py</code>) e dashboard HTML em tempo real	4

Tabela 1: Mapeamento dos objetivos específicos para os mecanismos de implementação.

3 Arquitetura

O sistema está organizado em quatro camadas com responsabilidades bem separadas, tal como ilustra a Figura 1. Esta secção fixa o vocabulário (nós, papéis, tópicos e mensagens) que a Secção 4 utiliza para detalhar a implementação.

3.1 Visão geral em camadas

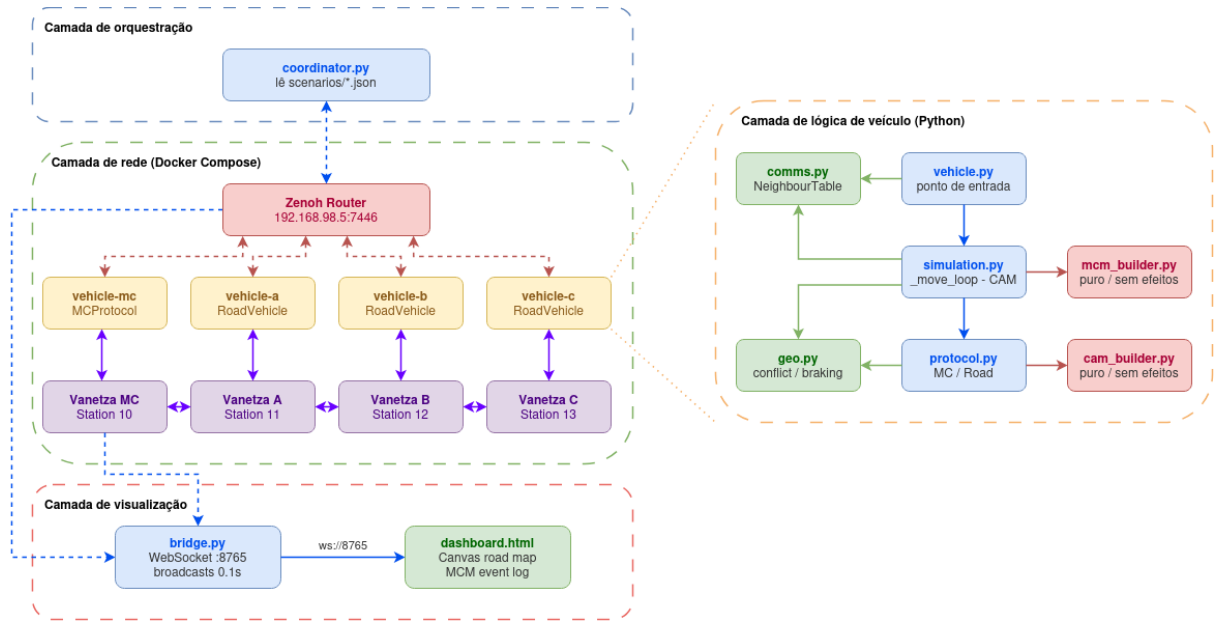


Figura 1: Arquitetura em camadas do sistema.

Camada de orquestração. Um único processo, o `coordinator.py` (GUI Tkinter), lê os ficheiros de cenário `cenarios/*.json`, publica o cenário escolhido no router Zenoh e aguarda que todos os veículos ativos reportem a conclusão. É o ponto de entrada da demonstração, mas *não* participa na negociação — apenas dá o pontapé de saída e recolhe o estado de cada veículo.

Camada de rede. Materializada por `docker-compose` sobre a rede Docker externa `vanetzalan0`. Contém o *router Zenoh* (canal do coordinator) e, por cada veículo, um par de containers: um container **Vanetza-NAP**, que implementa a pilha ETSI C-ITS (codificação/ decodificação ASN.1 e difusão pelo “ar”), e um container **Python**, que corre a lógica do veículo. É nesta camada que vivem as *stations* 10–13.

Camada de lógica de veículo (Python). É o cérebro de cada nó, organizado em módulos: `vehicle.py` (ponto de entrada), `simulation.py` (ciclo de movimento e emissão de CAM), `protocol.py` (a classe `MergeProtocol`, que concentra toda a negociação), apoiados por `geo.py` (geometria e modelo de travagem), `comms.py` (sessões Zenoh e `NeighbourTable`) e pelos construtores puros de mensagens `cam_builder.py` e `mcm_builder.py`. Esta camada é o objeto central da Secção 4.

Camada de visualização. O `bridge.py` subscreve o tráfego V2X (CAM e MCM) decodificado, mantém o estado de cada veículo e difunde, a cada 100 ms, uma trama JSON por WebSocket (porta 8765) para a `dashboard.html`, que desenha as estradas, os veículos e o registo de eventos MCM em tempo real.

Os dois planos de comunicação. A Figura 1 distingue, por cor, dois planos. O *plano V2X* (ligações entre cada veículo e o seu Vanetza e entre os Vanetza) transporta o tráfego C-ITS (CAM/MCM) e representa o meio rádio: na realidade, cada veículo difunde-o pelo broker Zenoh do seu *próprio* container Vanetza-NAP (`tcp/localhost:7447`) — não existe broker partilhado para o V2X, e a figura abstrai esses brokers como ligações diretas. O *plano de orquestração* (ligações ao bloco central “Zenoh Router”, `192.168.98.5:7446`) transporta apenas as mensagens de controlo da demonstração (cenário, *ready*, *done*). Assim, nenhum veículo comunica com outro por um canal partilhado privilegiado — toda a coordenação inter-veicular acontece exclusivamente através de mensagens C-ITS difundidas pelo respetivo Vanetza.

3.2 Nós e topologia

O cenário envolve **quatro veículos** — o Merge Car (MC) e os veículos da via principal A, B e C. Cada veículo é, na prática, um *par de containers* (Vanetza-NAP + Python) com uma identidade C-ITS fixa, resumida na Tabela 2. A estes junta-se o router Zenoh (canal de orquestração) e a cadeia de observabilidade — o `bridge.py` e a `dashboard` —, que apenas consome o tráfego decodificado para visualização, sem participar na negociação.

Veículo	Papel	Station ID	MAC	IP Vanetza
MC	Merge Car (rampa)	10	6e:06:e0:03:00:10	192.168.98.10
A	Via principal	11	6e:06:e0:03:00:11	192.168.98.11
B	Via principal	12	6e:06:e0:03:00:12	192.168.98.12
C	Via principal	13	6e:06:e0:03:00:13	192.168.98.13
Router Zenoh (coordinator)		—	—	192.168.98.5:7446

Tabela 2: Nós do sistema: identidade C-ITS e endereçamento. Cada veículo expõe ainda o broker Zenoh do seu Vanetza na porta 7447.

Cada container Python liga-se ao broker Zenoh do seu próprio Vanetza (`VANETZA_ZENOH_URL`) para o tráfego V2X e ao router central (`COORDINATOR_ZENOH_URL`) para o canal de orquestração. A geração nativa de CAM no Vanetza está **desativada** (`VANETZA_CAM_PERIODICITY=0`): toda a emissão de CAM é controlada pela camada Python, garantindo um *tick* de simulação coerente entre movimento e *beaconing*.

3.3 Mapa de tópicos Zenoh

A comunicação entre as camadas reduz-se a um pequeno conjunto de tópicos Zenoh (Tabela 3). Os tópicos `vanetza/in/*` e `vanetza/out/*` representam, respetivamente, a injeção de uma mensagem na pilha C-ITS e a sua receção após passar pelo “ar”; os tópicos `coordinator/*` constituem o canal de controlo da demonstração.

Tópico	Publica	Subscreve	Conteúdo
vanetza/in/cam	Veículo (Python)	Vanetza-NAP	CAM a difundir (dict JSON)
vanetza/out/cam	Vanetza-NAP	Veículos, <code>bridge</code>	CAM recebida do ar
vanetza/in/mcm	Veículo (Python)	Vanetza-NAP	MCM a difundir (dict JSON)
vanetza/out/mcm	Vanetza-NAP	Veículos, <code>bridge</code>	MCM recebida do ar
coordinator/scenario	<code>coordinator</code>	Veículos, <code>bridge</code>	Cenário completo (JSON)
coordinator/ready/<sid>	Veículo	<code>coordinator</code>	<i>Heartbeat</i> : veículo vivo
coordinator/done/<sid>	Veículo	<code>coordinator</code>	Veículo concluiu o cenário

Tabela 3: Mapa de tópicos Zenoh. <sid> é o *Station ID* do veículo.

3.4 Modelo de papéis

Uma decisão central da arquitetura é que **todos os veículos correm exatamente o mesmo código**. Não existe um binário “MC” e outro “veículo da estrada”: o papel *emerge em tempo de execução* do tipo de estrada onde o veículo é colocado pelo cenário. Um veículo posicionado numa estrada do tipo `ramp` com campo `merges_into` ativa o comportamento de Merge Car dentro do `MergeProtocol`; o mesmo código numa estrada `main` responde como veículo alvo da negociação. Esta unificação tem duas consequências importantes: (i) os cenários multi-MC são suportados sem código adicional — basta colocar dois veículos na rampa (ver cenários 04–06); e (ii) toda a lógica de negociação vive numa única classe, descrita em detalhe na Secção 4.

Coerentemente, **nenhum veículo tem conhecimento pré-configurado da sua vizinhança**. Quem está à frente, quem está atrás, quem entra em conflito e em que estrada cada um circula é inferido em tempo real a partir das posições GPS recebidas nas CAM e da geometria das estradas definida no cenário.

3.5 Tipos de mensagens trocadas

Apenas dois tipos de mensagem ETSI C-ITS são usados (Tabela 5). A **CAM** é o *beaconing* periódico que sustenta a consciência situacional; a **MCM** implementa o protocolo de negociação e desdobra-se em seis subtipos. Cada MCM transporta no `basicContainer` três campos que, em conjunto, identificam o subtipo: `mcmType` (tipo de mensagem), `itssRole` (papel do emissor) e `manoeuvreId` (identificador da sessão); o significado de cada valor consta da Tabela 4. Note-se em particular que o `manoeuvreId` — no standard, apenas um identificador de 1 byte — é usado, por convenção do projeto, para distinguir um `MERGE_GRANT` (0–127) de um `SLOWDOWN_GRANT` (128–255), que de outra forma seriam indistinguíveis.

Campo	Valor	Significado
mcmType	1	<i>request</i> — pedido de manobra
	2	<i>response</i> — resposta a um pedido
	7	<i>executionStatus</i> — estado de execução da manobra
itssRole	1	<i>coordinatingItss</i> — ITS-S que coordena o merge (o MC)
	3	<i>targetVehicle</i> — veículo-alvo (a quem se pede para abrandar)

Tabela 4: Significado dos códigos de `mcmType` e `itssRole` (enumerações ETSI) usados no `basicContainer`.

Subtipo	mcmType	itssRole	manoeuvreId	Direção	Propósito
MERGE_REQUEST	1	1	0–127	MC → todos	Pedir autorização; inclui zona de conflito e janela de ocupação
SLOWDOWN_REQUEST	1	3	0–127	em conflito → de trás	Propagar pedido de abrandamento na cadeia (via <code>executantID</code>)
SLOWDOWN_GRANT	2	3	128–255	atrás → à frente	Confirmar/recusar a aceitação do abrandamento
MERGE_GRANT	2	3	0–127	cada veículo → MC	Conceder a passagem ao MC, individualmente
MERGE_CONFIRMED	2	1	0–127	MC → todos	Confirmar (<i>agree</i>) ou abortar a manobra
EXECUTION_STATUS	7	1	0–127	MC → todos	MC já entrou na via; veículos retomam a velocidade

Tabela 5: CAM (*beaconing* periódico) e os seis subtipos de MCM do protocolo. O significado dos códigos de `mcmType`, `itssRole` e `manoeuvreId` consta da Tabela 4.

4 Implementação

Esta secção detalha o *cérebro* de cada nó — a camada de lógica de veículo em Python —, onde vive toda a negociação. Segue uma ordem *top-down*: começa por uma vista geral do código e por uma leitura da negociação de ponta a ponta (Secção 4.1); desce depois ao protocolo e às suas máquinas de estados (Secção 4.2); formaliza os fundamentos geométricos e cinemáticos que sustentam os cálculos (Secção 4.3); descreve o transporte de mensagens e a consciência situacional (Secção 4.4); e termina na orquestração e visualização (Secção 4.5).

4.1 Visão geral e a negociação num relance

Toda a lógica vive em `merge_lane/vehicle/`, repartida por módulos com responsabilidades estreitas:

- `vehicle.py` — ponto de entrada do container;
- `simulation.py` — o ciclo de movimento (`_move_loop`) e a emissão de CAM a cada *tick*;
- `protocol.py` — a classe `MergeProtocol`, que concentra *toda* a negociação (Secção 4.2);

- `geo.py` — geometria das estradas e modelo cinemático de travagem (Secção 4.3);
- `comms.py` — sessões Zenoh e a `NeighbourTable`;
- `cam_builder.py` e `mcm_builder.py` — construtores puros de mensagens (Secção 4.4).

Um pequeno conjunto de **constantes globais** (Tabela 6), definidas em `geo.py` e `protocol.py`, reaparece ao longo de toda a descrição e fixa o comportamento temporal e cinemático do sistema.

Constante	Valor	Papel
DT	0,1 s	período do <i>tick</i> de simulação (e de CAM)
ACCEL = DECEL	4,0 m/s ²	taxa de aceleração e de travagem
SAFETY_GAP	10 m	folga de segurança nas margens da zona de conflito
CONFLICT_HORIZON_S	4 s	ETA ao <i>merge point</i> a que o MC inicia a negociação
MERGE_REQUEST_RESEND_S	1 s	intervalo mínimo de reenvio do MERGE_REQUEST
RETRY_COOLDOWN_S	1 s	espera após um <i>abort</i> antes de nova tentativa

Tabela 6: Constantes globais usadas ao longo da implementação.

Uma decisão estruturante — detalhada na Secção 4.2 — é que **todos os veículos correm exatamente o mesmo código**: o papel (Merge Car ou veículo da via principal) *emerge em tempo de execução* do tipo de estrada onde o veículo é colocado, e nenhum veículo tem conhecimento pré-configurado da sua vizinhança.

A negociação num relance. Antes de descer aos detalhes, a Figura 2 dá a vista de conjunto do protocolo num cenário concreto: o MC funde-se na via onde circulam A, B e C (A à frente, C atrás), estando apenas **B em conflito**. A sequência decorre como se segue (os subtipos de MCM estão definidos na Tabela 5):

1. **Pedido.** Ao atingir o horizonte de 4 s, o MC difunde MERGE_REQUEST a todos (A, B, C), com a zona de conflito e a janela de ocupação.
2. **Grants diretos.** A e C avaliam o conflito e, não se sobrepondo à janela do MC, concedem MERGE_GRANT de imediato.
3. **Conflito e cadeia.** B deteta sobreposição, calcula a velocidade-alvo e, tendo C atrás de si, envia-lhe SLOWDOWN_REQUEST em vez de responder já ao MC.
4. **Retorno da cadeia.** C (fim de cadeia) confirma que trava a tempo e devolve SLOWDOWN_GRANT a B; B valida a própria travagem e só então envia o seu MERGE_GRANT ao MC.
5. **Confirmação.** Com `active_peers = {A, B, C} ⊆ grants_received`, o MC difunde MERGE_CONFIRMED(agree). B (e a sua cadeia) aplica então o abrandamento; o MC executa a inserção.
6. **Retoma.** Já na via principal, o MC emite EXECUTION_STATUS e todos os veículos retomam a velocidade normal.

Ramo alternativo (ABORT). Se C não conseguir travar a tempo, devolve `SLOWDOWN_GRANT(recusar)`; B propaga a recusa como `MERGE_GRANT(recusar)` ao MC. O MC reage com `MERGE_CONFIRMED(abort)`, repõe o estado e volta a tentar após o *cooldown* de 1 s — ou, se entretanto alcançar o `stop_t`, recorre ao *stop-and-wait* (Secção 4.2.1).

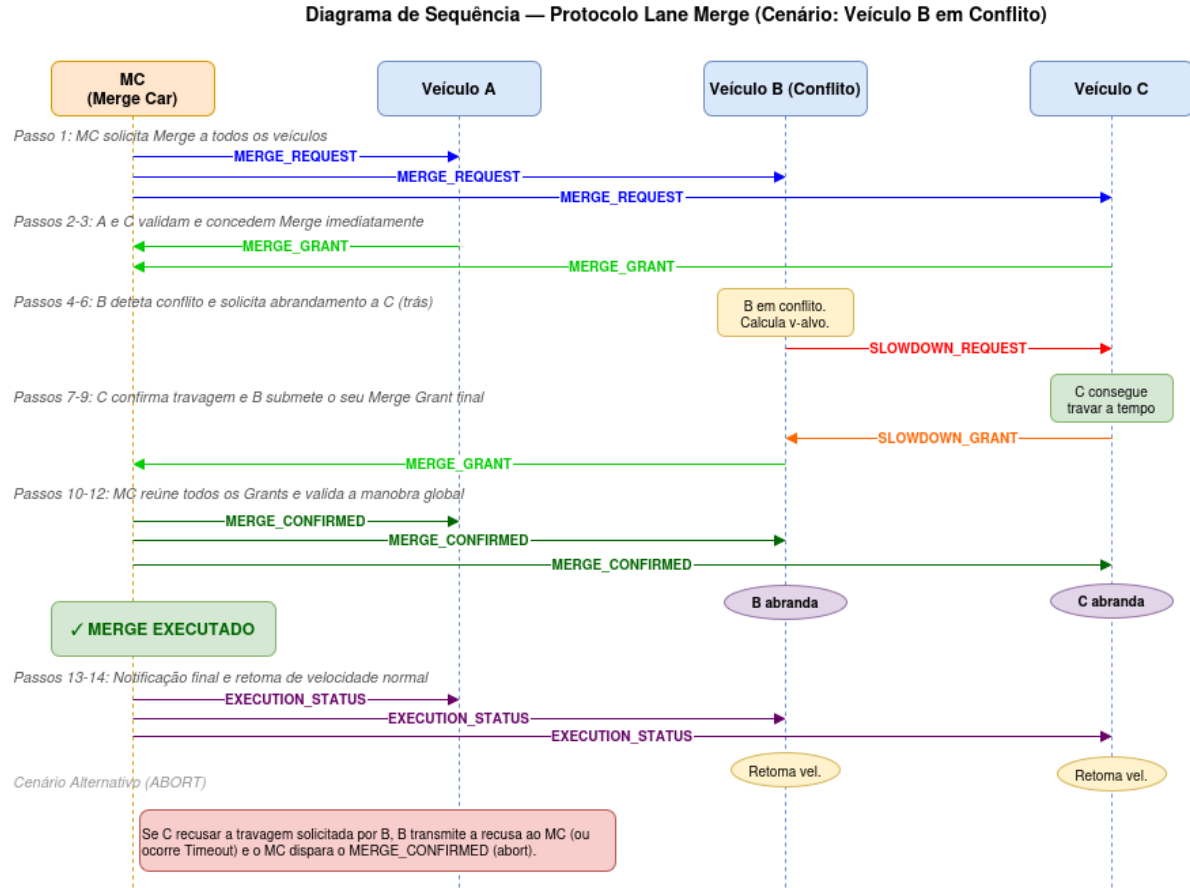


Figura 2: Diagrama de sequência do protocolo (cenário com o veículo B em conflito).

4.2 O protocolo MergeProtocol

Toda a negociação está numa única classe, `MergeProtocol`, partilhada por todos os veículos (O2, O6). O papel *emerge* em tempo de execução: `set_current_road` liga `current_is_ramp` quando a estrada é do tipo `ramp` com `merges_into`, e é esse *flag* (`_is_ramp()`) que encaminha cada veículo para o comportamento de Merge Car (Secção 4.2.1) ou de veículo da via principal (Secção 4.2.2). O estado é protegido por um `threading.Lock`, pois o *tick* de movimento (uma *thread*) e os *callbacks* MCM (outra) acedem-no concorrentemente. A interface pública resume-se a `tick(...)`, chamado a cada *tick* e que devolve `{advance, target_speed}` ao ciclo de movimento, e `make_mcm_callback()`, que despacha cada MCM recebida pela tripla (`mcmType`, `itssRole`, `manoeuvreId`) (cf. Tabela 5). As duas subsecções seguintes descrevem os dois comportamentos que esta classe encerra.

4.2.1 Lado do Merge Car (rampa)

A Figura 3 resume os estados do Merge Car; descrevem-se a seguir. O comportamento concentra-se em `_tick_ramp`, executado a cada *tick*.

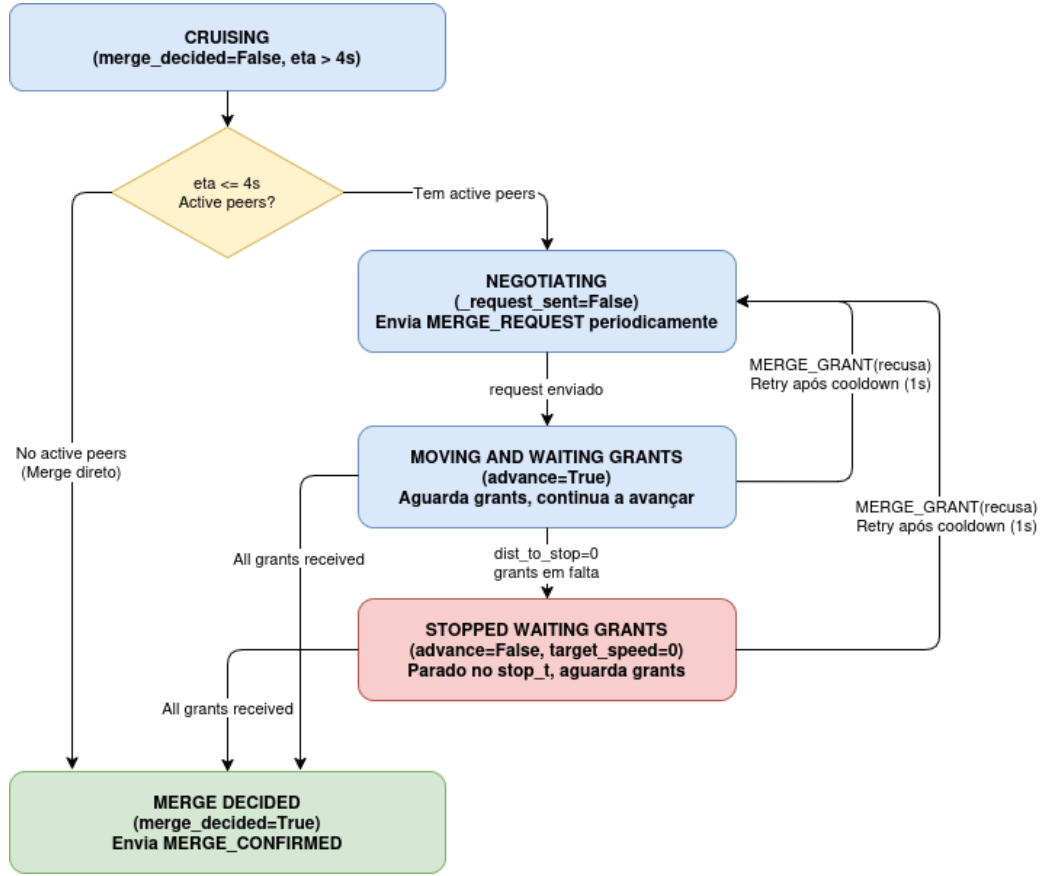


Figura 3: Máquina de estados do Merge Car.

Enquanto a velocidade é positiva, o MC estima o tempo até ao *merge point* com base na fração restante da rampa, recorrendo ao modelo cinemático (Secção 4.3):

$$\text{eta_s} = \text{time_to_cover_distance}(v, v_{\text{rampa}}, (1 - t) L_{\text{rampa}}).$$

Em **CRUISING**, enquanto $\text{eta_s} > \text{CONFLICT_HORIZON_S} = 4\text{s}$, o MC limita-se a circular. Quando o ETA desce ao horizonte de 4s, avalia os *active peers* — os vizinhos que *is_on_road* (Secção 4.3) coloca na via principal:

- **sem peers ativos** → não há com quem negociar: o MC salta direto para `_on_all_granted` (*merge direto*);
- **com peers** → entra em **NEGOTIATING** e chama `_maybe_send_merge_request`.

Construção do `merge_request`. O MC anuncia dois elementos. Primeiro, a *zona de conflito* — o troço que vai ocupar, cujo início e fim já estão fixados (Secção 4.3) e viajam nos *waypoints* do TRR como *offsets* relativos à sua posição. Segundo, a *janela de ocupação* dessa zona: estima a velocidade de entrada na via, $\text{entry_speed} = \text{predict_motion}(v, v_{\text{rampa}}, \text{eta_s})$, e depois

quanto tempo a ocupará, `occupancy = time_to_cover_distance(entry_speed, vmain, after_m)` (ambas do modelo cinemático, Secção 4.3); a janela `[eta_s, eta_s + occupancy]` (em ms) viaja nos campos `tRROccupancyStartTime/EndTime`. Cada envio incrementa `manoeuvre_id` de forma cíclica em `[0, 127]` `((id+1)%128)` e é limitado a um por `MERGE_REQUEST_RESEND_S=1 s`.

Recolha de *grants* e decisão. Cada `MERGE__GRANT(aceitar)` recebido acrescenta o emissor ao conjunto `grants_received` (estado **MOVING AND WAITING GRANTS** — o MC continua a avançar enquanto espera). Quando `active_peers ⊆ grants_received`, `_on_all_granted` marca `merge_decided` e difunde `MERGE__CONFIRMED(agree)`, transitando para **MERGE DECIDED**. Se em vez disso chegar um `MERGE__GRANT(recusar)` (`on_merge_grant` com `success=False`), o MC limpa os *grants*, difunde `MERGE__CONFIRMED(abort)` e agenda nova tentativa após `RETRY_COOLDOWN_S=1 s`.

Salvaguarda *stop-and-wait* e cadeia na rampa. Em paralelo, o MC monitoriza a distância ao `stop_t` (Secção 4.3). Se `can_brake_in_time` indicar que está prestes a alcançá-lo sem ter decidido, passa o alvo a `target_speed=0` — o que o faz travar progressivamente (à taxa **DECEL**) — e, ao imobilizar-se, fica **STOPPED WAITING GRANTS** (`advance=False`), continuando a renegociar até obter todos os *grants* (O5). Nesse estado, e relevante para cenários multi-MC (O6), `_maybe_send_ramp_slowdown` procura um veículo atrás *na própria rampa* (`find_vehicle_behind`) e propaga-lhe um `SLOWDOWN__REQUEST` — a mesma cadeia de abrandamento da via principal, agora aplicada na rampa.

4.2.2 Lado do veículo da via principal

A Figura 4 resume os estados de um veículo da via principal; descrevem-se a seguir.

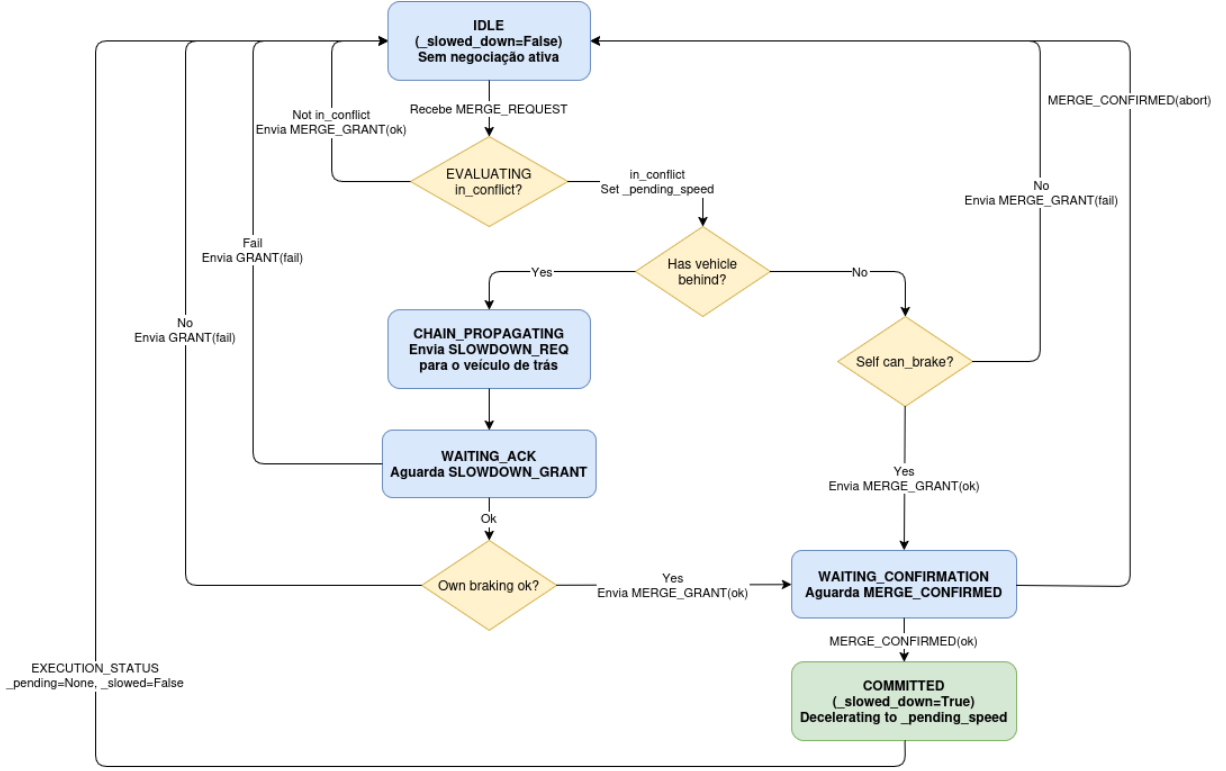


Figura 4: Máquina de estados do veículo na via principal.

Um veículo da via principal está normalmente em **IDLE**. Ao receber um `MERGE_REQUEST`, entra em `_on_merge_request` e **avalia o conflito**.

Deteção de conflito (O3). O veículo reconstrói a zona a partir dos *waypoints* do TRR (somando os *offsets* à posição do MC) e projeta-a na sua própria estrada com `project_t` (Secção 4.3), obtendo `cz_t_start` e `cz_t_end`. Calcula então as distâncias até entrar e sair da zona (descontando/somando a meia-carroçaria), converte-as nos instantes `t_enter` e `t_exit` a velocidade constante, e declara conflito se essa janela se **sobrepe**õe à janela de ocupação anunciada pelo MC, `[mc_eta_start, mc_eta_end]` (Algoritmo 1).

Algorithm 1: Deteção de conflito (`_on_merge_request`)

Input: posição t , velocidade v , zona $[cz_t_start, cz_t_end]$, janela do MC $[eta_s, eta_e]$, comprimento L_{veh} , L

Output: $in_conflict \in \{V, F\}$

- 1 $d_{enter} \leftarrow \max(0, (cz_t_start - t) L - L_{veh}/2);$
 - 2 $d_{exit} \leftarrow \max(0, (cz_t_end - t) L + L_{veh}/2);$
 - 3 $t_{enter} \leftarrow \text{time_to_cover_distance}(v, v, d_{enter});$
 - 4 $t_{exit} \leftarrow \text{time_to_cover_distance}(v, v, d_{exit});$
 - 5 **return** $(t_{enter} < eta_e) \wedge (t_{exit} > eta_s);$
-

Sem conflito. O veículo responde de imediato com `MERGE_GRANT(aceitar)` ao MC e regressa a **IDLE** — não precisa de abrandar.

Com conflito: velocidade-alvo (O4). Detetado o conflito, o veículo tem de abrandar o suficiente para não chegar à zona enquanto o MC a ocupa, mas sem travar mais do que o necessário. `_solve_target_speed` resolve esse compromisso por busca binária (24 iterações): procura a *maior* velocidade-alvo v^* tal que, abrandando para ela, o veículo percorre *menos* do que a distância d que o separa do início da zona durante o intervalo `eta` do MC — ou seja, no instante em que o MC ocupa a zona, este veículo ainda não lá chegou. A previsão a cada passo usa o modelo cinemático real (`predict_motion`, com aceleração limitada, Secção 4.3), pelo que o v^* encontrado é mesmo atingível; maximizá-lo minimiza o abrandamento imposto e preserva a fluidez (Algoritmo 2). A velocidade-alvo fica guardada em `_pending_speed`.

Algorithm 2: Velocidade-alvo (`_solve_target_speed`)

Input: distância à zona d , ETA do MC `eta`, velocidade atual v , limite $v_{\max}$

Output: velocidade-alvo

```

1 if eta ≤ 0 ou  $d \leq 0$  then return 0;
2  $lo \leftarrow 0$ ;  $hi \leftarrow \max(v, v_{\max})$ ;
3 for 24 vezes do
4    $mid \leftarrow (lo + hi)/2$ ;
5    $(d_{\text{pred}}, \_) \leftarrow \text{predict\_motion}(v, mid, \text{eta})$ ;
6   if  $d_{\text{pred}} > d$  then  $hi \leftarrow mid$ ;
7   else  $lo \leftarrow mid$ ;
8 return  $hi$ ;
```

Propagação em cadeia (O4). Com a velocidade-alvo calculada, o veículo procura um veículo imediatamente atrás (`find_vehicle_behind`, Secção 4.3):

- **há veículo atrás** $\rightarrow$ propaga a cadeia (**CHAIN_PROPAGATING**): envia-lhe `SLOWDOWN_REQUEST` com a sua sugestão e fica em **WAITING_ACK** à espera do `SLOWDOWN_GRANT`;
- **não há veículo atrás** (fim da cadeia) $\rightarrow$ verifica a própria viabilidade com `can_brake_in_time` (Secção 4.3): se consegue travar, compromete-se (`_slowed_down = True`) e envia `MERGE_GRANT`(aceitar); senão, `MERGE_GRANT`(recusar).

`_on_slowdown_request` replica esta lógica num veículo intermédio: adota a velocidade sugerida (ou 0,7 v por omissão), e ou reenvia `SLOWDOWN_REQUEST` ao seguinte, ou — se for o fim da cadeia — testa a sua travagem e responde com `SLOWDOWN_GRANT`. Quando o `SLOWDOWN_GRANT` *regressa* pela cadeia, `_on_slowdown_grant` confirma que a *própria* travagem continua viável e propaga-o para a frente; o veículo no topo de cada cadeia (sem remetente à frente) traduz esse grant final no `MERGE_GRANT` que envia ao MC. Assim, um veículo em conflito só concede a passagem ao MC depois de garantir que toda a cadeia atrás de si consegue abrandar em segurança (Algoritmo 3).

Algorithm 3: Propagação em cadeia do abrandamento

Input: velocidade-alvo v_{alvo} , remetente à frente (ou MC)

```
1  $\_pending\_speed \leftarrow v_{\text{alvo}}$ ;
2  $b \leftarrow \text{find\_vehicle\_behind}(\dots)$ ;
3 if  $b \neq \emptyset$  then
4   enviar SLOWDOWN_REQUEST( $b, v_{\text{alvo}}$ );
5   aguardar SLOWDOWN_GRANT de  $b$ ; //  $\_on\_slowdown\_grant$ 
6 else
7    $(\text{ok}, d_{\text{trav}}) \leftarrow \text{can\_brake\_in\_time}(v, v_{\text{alvo}}, d_{\text{disp}})$ ;
8 if cadeia atrás aceitou e própria travagem ok then
9    $\_slowed\_down \leftarrow \text{Verdadeiro}$ ;
10  responder GRANT(aceitar) ao remetente à frente ou MERGE_GRANT ao MC;
11 else
12   $\_pending\_speed \leftarrow \emptyset$ ;
13  responder GRANT(recusar) / MERGE_GRANT(recusar);
```

Execução e retoma. O $_pending_speed$ é apenas *pendente*: só é efetivamente aplicado (passa a **COMMITTED**) quando chega MERGE_CONFIRMED(agree) em $_on_merge_confirmed$ — a partir daí, $\text{tick}()$ devolve essa velocidade ao ciclo de movimento e o veículo desacelera. Um MERGE_CONFIRMED(abort) cancela o abrandamento e devolve-o a **IDLE**. Por fim, ao receber EXECUTION_STATUS ($_on_execution_status$, o MC já entrou na via), limpa $_slowed_down/_pending_speed$ e retoma a velocidade-limite da estrada.

4.3 Fundamentos geométricos e cinemáticos

Esta secção formaliza as primitivas de `geo.py` que sustentam os cálculos do protocolo (Secção 4.2): traduzem as posições GPS das CAMs em quantidades operacionais e modelam a dinâmica de travagem. Todas as estradas são segmentos de reta definidos no cenário por **start** e **end**.

Distância e projeção. A distância métrica entre dois pontos GPS usa a fórmula de *haversine* (`haversine`, $R = 6\,371\,000$ m):

$$d = 2R \arcsin \sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}}.$$

A posição de um veículo ao longo de uma estrada é dada por uma **coordenada paramétrica** t , obtida pela projeção escalar do ponto sobre o segmento (`project_t`), com $t = 0$ no início e $t = 1$ no fim:

$$t = \frac{(P - s) \cdot (e - s)}{\|e - s\|^2}, \quad s = \text{start}, e = \text{end},$$

calculada diretamente em graus (a anisotropia lat/lon é desprezável à escala local). O comprimento físico do segmento, $L = \text{road_length}$, converte t em metros ($\text{dist} = \Delta t \cdot L$). O rumo do segmento vem de `compute_bearing` (azimute referenciado a Norte). Esta projeção é usada pelo veículo da via principal para reposicionar a zona de conflito na sua estrada (Secção 4.2.2).

Pertença a uma estrada. Como na simulação os veículos se movem por interpolação exata sobre a reta, a distância entre a posição GPS e a sua projeção é ~ 0 m; `is_on_road` considera um veículo “nesta estrada” se essa distância for inferior a um limiar de 0,05 m (apenas para absorver erro de vírgula flutuante). É este teste que permite ao MC distinguir, em tempo real, quais dos vizinhos circulam na via principal (os *active peers*, Secção 4.2.1).

Vizinho de trás e movimento. `find_vehicle_behind` percorre o *snapshot* da `NeighbourTable` (Secção 4.4) e devolve o veículo com o maior $t_n < t_{\text{próprio}}$ que esteja na mesma estrada e dentro de 200 m — a base da propagação em cadeia (Secção 4.2.2). O movimento de cada veículo é um avanço incremental ao longo do segmento, $t += \text{speed} \cdot \text{DT}/L$, executado a cada *tick* pelo `_move_loop`.

Dinâmica longitudinal. A dinâmica é deliberadamente simples e determinística, com aceleração limitada e simétrica $\text{ACCEL} = \text{DECEL} = 4,0 \text{ m/s}^2$. `advance_speed` aproxima a velocidade do alvo em cada *tick*, no máximo $\pm \text{DECEL} \cdot \text{DT}$, devolvendo também a aceleração efetiva (que é depois anunciada na CAM). Sobre esta primitiva constroem-se duas funções de previsão usadas na negociação:

- `predict_motion(v, v_alvo, horizonte)` — integra `advance_speed` passo a passo e devolve a distância percorrida e a velocidade final, respeitando o limite de aceleração (não é cinemática de velocidade constante); base da velocidade-alvo e da janela de ocupação (Secções 4.2.1 e 4.2.2);
- `time_to_cover_distance(v, v_alvo, d)` — o inverso: o tempo necessário para cobrir d , ou ∞ se for impossível (e.g. parado com alvo nulo); base do ETA e dos instantes de entrada/saída na deteção de conflito.

Viabilidade de travagem. `can_brake_in_time` usa a equação de Torricelli para verificar se um veículo consegue desacelerar de v até v_{alvo} dentro da distância disponível:

$$d_{\text{trav}} = \frac{v^2 - v_{\text{alvo}}^2}{2 \cdot \text{DECEL}}, \quad \text{aceita se } d_{\text{trav}} \leq d_{\text{disp}}.$$

Este é o teste de segurança (O5) aplicado por cada veículo no fim de cadeia antes de se comprometer com um abrandamento (Secção 4.2.2) e pelo MC antes do *stop-and-wait* (Secção 4.2.1).

Dimensionamento da zona de conflito. A zona de conflito é o troço da via principal que o MC irá ocupar. As suas margens (em metros), fixadas em `_init_merge_geometry`, refletem o espaço físico que a manobra exige:

$$\text{before_m} = \frac{L_{\text{veh}}}{2} + \text{SAFETY_GAP} + \underbrace{\frac{v_{\text{main}}^2 - v_{\text{mc}}^2}{2 \cdot \text{DECEL}}}_{\text{merge_entry_margin}}, \quad \text{after_m} = \frac{L_{\text{veh}}}{2} + \text{SAFETY_GAP}.$$

A meia-carroçaria $L_{\text{veh}}/2$ e o `SAFETY_GAP` = 10 m são comuns aos dois lados; a montante acrescenta-se ainda a `merge_entry_margin`, que é precisamente a distância de travagem necessária para um veículo da via principal (a v_{main}) igualar a velocidade de entrada do MC (a v_{mc}) —

nula quando $v_{\text{main}} \leq v_{\text{mc}}$. Esta assimetria reserva mais espaço atrás do *merge point*, onde o diferencial de velocidade é maior. A partir destas margens, `conflict_zone_t` exprime a zona em coordenadas paramétricas, centrada no *merge point* t_{merge} :

$$t_{\text{start}} = t_{\text{merge}} - \frac{\text{before_m}}{L}, \quad t_{\text{end}} = t_{\text{merge}} + \frac{\text{after_m}}{L}.$$

É esta zona que o MC codifica nos *waypoints* do `MERGE_REQUEST` (Secção 4.4) e que cada veículo da via principal reprojecta na sua estrada (Secção 4.2.2).

Ponto de paragem do MC. Como salvaguarda (O5), o MC nunca ultrapassa um ponto da rampa sem ter decidido o *merge*. `mc_stop_t` fixa essa posição paramétrica deixando a frente do veículo a um *buffer* de 8 m do fim da rampa:

$$\text{stop_t} = 1 - \frac{L_{\text{veh}} + 8}{L_{\text{ramp}}}.$$

4.4 Mensagens e consciência situacional

Esta secção descreve como as mensagens são construídas, transportadas e recebidas, e como delas emerge a consciência situacional que alimenta todo o protocolo.

Ciclo de vida de uma mensagem. O ciclo é uniforme para CAM e MCM e atravessa as quatro camadas da Secção 3. Do lado emissor, um construtor puro devolve um dicionário: `build_cam` é invocado a cada *tick* pelo ciclo de simulação (`simulation._move_loop`), enquanto os `build_*` de `mcm_builder.py` são chamados pela classe `MergeProtocol` (Secção 4.2) quando a negociação o exige. Esse dicionário é serializado em JSON e publicado no tópico de *entrada* da pilha C-ITS:

```
vanetza_session.put("vanetza/in/cam", json.dumps(cam).encode())
```

O container Vanetza-NAP subscreve `vanetza/in/*`, aplica as escalas inteiras do *codec*, codifica a PDU em ASN.1/UPER e difunde-a pelo “ar”. Do lado recetor, a PDU volta a ser decodificada e republicada em `vanetza/out/*`, onde os *callbacks* de cada veículo (`make_cam_callback`, `MergeProtocol.make_mcm_callback`) e o `bridge.py` a consomem. O *payload* decodificado chega encapsulado em `payload["fields"]` — sob a chave `cam` para CAM e `payload` para MCM —, e cada *callback* começa por descartar a mensagem se o `stationID` for o seu próprio.

Beaconing e NeighbourTable (O1). A consciência situacional assenta num *beaconing* CAM síncrono com o movimento. Em cada *tick* de $DT = 0,1s$, o `_move_loop` calcula a posição interpolada, atualiza a velocidade e constrói uma CAM com `build_cam` (posição, rumo, velocidade, aceleração do *tick* e comprimento), publicando-a em `vanetza/in/cam`. Como a geração nativa de CAM no Vanetza está desativada (`VANETZA_CAM_PERIODICITY=0`), há exatamente uma CAM por *tick* de simulação, garantindo coerência temporal entre o que cada veículo *faz* e o que *anuncia*. Do lado da receção, `make_cam_callback` extrai do `basicVehicleContainerHighFrequency` a posição, velocidade, rumo e comprimento e invoca `NeighbourTable.update`. A `NeighbourTable`

(em `comms.py`) é uma tabela *thread-safe* que guarda apenas o estado mais recente de cada vizinho, carimbado com o instante de receção. O método central é `snapshot(max_age_s=0.2)`, usado por toda a lógica de protocolo: devolve somente as entradas recebidas nos últimos 200 ms. Com CAMs a cada 100 ms, esta janela tolera exatamente *uma* CAM perdida antes de um vizinho “desaparecer” da vista, evitando decisões sobre informação obsoleta. Assim, **toda** a noção de vizinhança — quem existe, onde está, a que velocidade — é derivada exclusivamente das CAMs recebidas, sem qualquer configuração prévia.

Construtores de MCM. `mcm_builder.py` contém **funções puras** (sem I/O): recebem o estado do veículo e devolvem o dicionário da PDU pronto a serializar; a decisão de *quando* e *a quem* enviar vive em `protocol.py` (Secção 4.2). Toda a MCM começa por um `basicContainer` comum — com o `mcmType`, o `itssRole`, o `manoeuvreId` e a posição do emissor — cuja tripla (`mcmType`, `itssRole`, `manoeuvreId`) identifica o subtipo (Tabela 5). O que vem a seguir distingue um *pedido* de uma *resposta*:

- **Pedidos** (`MERGE_REQUEST`, `SLOWDOWN_REQUEST`) descrevem a manobra. Levam o estado cinemático do emissor (velocidade, rumo, dimensões) e, sobretudo, uma lista `manoeuvreAdvice` em que *cada entrada endereça um destinatário* pelo seu `executantID` (o *stationID*) e lhe sugere uma velocidade — um recetor só atua sobre a entrada cujo `executantID` é o seu. O `MERGE_REQUEST` acrescenta o *Target Road Resource* (TRR), que transporta a zona de conflito: início e fim (Secção 4.3) codificados como *offsets* (Δlat , Δlon) relativos à posição do MC, o comprimento da zona e a janela de ocupação.
- **Respostas** (`MERGE_GRANT`, `SLOWDOWN_GRANT`, `MERGE_CONFIRMED`, `EXECUTION_STATUS`) são mínimas: resumem-se a um `responseContainer` com um único campo `manoeuvreResponse` (0 = aceitar/acordar, 1 = recusar/abortar).

Em esquema, um `MERGE_REQUEST` do MC (*station* 10) que pede a A (*station* 11) para abrandar para 15 m/s tem a forma:

```

1 basicContainer:      { mcmType=1, itssRole=1, manoeuvreId=7, lat, lon }   # pedido, MC (
   coordinator)
2 mcmContainer.vehicleManoeuvreContainer:
3   vehicleCurrentStateContainer: { speed, heading, length }              # estado do MC
4   targetRoadResource: zona de conflito = offsets(inicio, fim) + comprimento + janela
5   manoeuvreAdvice: [ { executantID=11, advisedSpeed=15 m/s } ]          # destinatario =
   A

```

`MERGE_GRANT` e `SLOWDOWN_GRANT` partilham `mcmType=2` e `itssRole=3`; distinguem-se apenas pelo intervalo do `manoeuvreId` (1 byte): **0–127** para o primeiro e **128–255** para o segundo.

4.5 Orquestração, visualização e modo demo

Em torno da lógica de veículo existem componentes de suporte que não participam na negociação.

Coordenação (`coordinator.py`). Uma GUI Tkinter lê os cenários de `scenarios/*.json`, e ao seleccionar um publica-o em `coordinator/scenario` no router Zenoh. Cada container de veículo subscreve esse tópico e, se o seu `station_id` constar de `active_vehicles`, corre o cenário,

emitindo *heartbeats* em `coordinator/ready/<sid>` e, no fim, `coordinator/done/<sid>`. O coordenador aguarda o *done* de todos antes de avançar. Expõe ainda dois interruptores: **Demo Mode** (ativa o `DemoMergeProtocol`, descrito abaixo) e **Draw Exag** (fator de exagero visual enviado à *dashboard*). O `run.sh` encadeia todo o arranque: `docker-compose up`, *bridge*, servidor HTTP, a GUI e o browser.

Visualização (`bridge.py` + `dashboard.html`) (O7). O `bridge.py` é o único componente que fala simultaneamente Zenoh e WebSocket. Subscribe `vanetza/out/{cam,mcm}` e `coordinator/scenario`, mantém o estado de cada veículo e difunde uma trama JSON a cada 100 ms pela porta 8765 para a `dashboard.html` (*canvas* HTML, sem mapas). A função `_classify_mcm` traduz cada MCM num rótulo legível (e.g. `MERGE_REQUEST`, `SLOWDOWN_GRANT`), e o estado visual de cada veículo (`NORMAL`, `SLOWING`, `SPEEDING`, `MERGING`) é inferido do sinal da aceleração anunciada na CAM e das transições `MERGE_CONFIRMED/EXECUTION_STATUS`. O *merge point* é recalculado geometricamente por `_compute_merge_point` (interseção das retas *main* e *ramp*), e a zona de conflito é reconstruída a partir dos *waypoints* do `MERGE_REQUEST`.

Modo Demo (`DemoMergeProtocol`). Para tornar a negociação observável passo a passo, `protocol_demo.py` define `DemoMergeProtocol`, uma subclasse de `MergeProtocol`. A ideia é introduzir pausas *apenas* nos pontos de decisão relevantes: ao receber uma MCM, o *callback* determina se este veículo a vai realmente processar (`will_process`) e, nesse caso, dorme `demo_pause_s` antes de delegar no *callback* do progenitor. Os veículos da via principal congelam o movimento (`_demo_paused`) ao verem o `MERGE_REQUEST` e retomam-no ao `MERGE_CONFIRMED`. Para acomodar estas pausas, o reenvio do `MERGE_REQUEST` é limitado a `_demo_resend_s = 1 + 4 · demo_pause_s`, em vez do 1 s do modo normal.

5 Fluxo da Demonstração

5.1 Preparação

[**TODO:** `run.sh`; escolher cenário no `coordinator`; `dashboard` aberta.]

5.2 Timeline

[**TODO:** Ponto de partida → `MERGE_REQUEST` ao atingir o horizonte de 4 s → decisões em função das mensagens recebidas → abrandamento/merge → retoma de velocidade. Como termina a demo.]

5.3 Aleatoriedade e variáveis

[**TODO:** Aleatório: `manoeuvreId` do `SLOWDOWN_GRANT` (128–255), jitter de rede e ordem de chegada de CAMs/MCMs. Controlável: cenário (posições iniciais), Demo Mode + pausa, Draw Exag, speed limits e cumprimentos no JSON.]

5.4 Cenários

[**TODO:** Tabela dos 6 cenários: 01 standard; 02 colisão B; 03 colisão A+B+C; 04 2-MC standard; 05 2-MC colisão B; 06 2-MC colisão todos — com o resultado esperado de cada.]

6 Resultados

[**TODO:** Por cenário: comportamento observado (conflito, cadeia de abrandamento, merge sem colisão) com evidência (screenshots da dashboard + logs anotados). Validação dos casos de colisão (sem protocolo colidiria; com protocolo é seguro). Caso de abort + retry. Caso fallback stop-and-wait. Multi-MC (cadeia também na rampa). Discussão: limitações e simplificações do modelo.]

7 Conclusão

[**TODO:** Síntese do alcançado vs objetivos; lições aprendidas; trabalho futuro.]

Referências